SuCo as a FAISS Index

Dimitra Seferiadi
AM: 03120131

1 Introduction and Background

1.1 The Problem of High-Dimensional Approximate Nearest Neighbor Search

Nearest-neighbor (NN) search in high-dimensional Euclidean spaces is a fundamental primitive in machine learning and information retrieval, underlying applications as diverse as semantic search, image retrieval, and recommender systems. Exact NN search becomes prohibitively expensive as the number of dimensions grows, a phenomenon known as the *curse of dimensionality*, motivating Approximate Nearest Neighbor (ANN) search, which trades a small reduction in recall for large gains in query speed.

Modern ANN methods fall into four principal families: Locality-Sensitive Hashing (LSH) [1], Vector Quantization (VQ) [2], tree-based methods [3], and graph-based methods [4]. LSH-based methods offer rigorous probabilistic guarantees but suffer from high query latency and large memory footprints. Graph-based methods such as HNSW achieve state-of-the-art query throughput but require expensive index construction and large memory, and provide no theoretical quality guarantees. No existing approach simultaneously excels at indexing time, query speed, memory efficiency, and theoretical guarantees.

1.2 The SuCo Framework

The paper *"Subspace Collision: An Efficient and Accurate Framework for High-dimensional Approximate Nearest Neighbor Search"* [5] addresses this gap by introducing the *Subspace Collision (SC)* framework and its efficient instantiation, SuCo.

Core Intuition. Two data points that are close in the full d-dimensional space are, on average, also close when projected onto a random s-dimensional subspace. Rather than measuring the exact distance within a single subspace, which is noisy, the authors count how many out of N_s independent subspaces report the two points as close. A point earns a *collision* in subspace i if it ranks among the $\alpha \cdot n$ nearest neighbors of the query in that subspace; its *SC-score* is the total collision count across all N_s subspaces.

Formally, given n database points in $\mathbb{R}^d$ and a query q, the SC-score of a point o is

$$\text{SC-score}(o) = \sum_{i=1}^{N_s} \mathbf{1}\big[o \text{ is an } (\alpha n)\text{-NN of } q \text{ in subspace } S_i\big].$$

The authors demonstrate that SC-score follows a *Pareto principle*: the 20% of data points closest to the query concentrate most of the SC-score mass, while the remaining 80% have near-zero scores. This sharp separability makes SC-score an effective and theoretically grounded proxy for Euclidean distance.

Subspace Sampling. The d dimensions are partitioned without replacement into N_s non-overlapping subspaces $S_1, \ldots, S_{N_s}$, each of dimension $s = d/N_s$. This design ensures that every dimension contributes to exactly one subspace, maximising the information content of each collision.

The SuCo Index. The naive algorithm (SC-Linear) calculates exact distances in every subspace for every database point, yielding a complexity equivalent to linear scan. SuCo replaces this with an Inverted Multi-Index (IMI) [6] per subspace to accelerate collision counting. Each s-dimensional subspace is further halved into two $s/2$-dimensional parts, and K-means clustering with $n_c = \sqrt{K}$ centroids is run independently on each half. The resulting $K = n_c^2$ cells per subspace form the IMI.

At query time, for each subspace the algorithm computes distances to the $2n_c$ half-space centroids, then retrieves IMI cells in ascending order of their combined centroid distance using the *Dynamic Activation* algorithm (Algorithm 3 in the paper). Dynamic Activation avoids the logarithmic overhead of a priority queue by maintaining an explicit activation list, achieving up to 40% higher throughput than the classic Multi-sequence algorithm [6] while returning identical results.

After collision counting across all N_s subspaces, the top-$(\beta \cdot n)$ points by SC-score form the re-ranking candidate pool, from which exact Euclidean distances are computed and the top-k results returned.

Theoretical Guarantees. Theorems 1 and 2 in the paper provide formal guarantees:

- **Theorem 1** (Effectiveness of SC-score): Given two independent random data points o_1, o_2, if SC-score(o_1) > SC-score(o_2) then $\|o_1, q\| < \|o_2, q\|$ with probability at least $1/2 - 1/e^2 \approx 0.365$, for appropriate N_s and α.
- **Theorem 2** (Quality Guarantee): Algorithm 1 (SC-Linear) answers a k-ANN query correctly with probability at least 1/2, for appropriate choices of N_s, α, β.

These guarantees distinguish SuCo from graph-based methods, which provide no formal quality bounds.

2 FAISS-Based Implementation

2.1 Overview of FAISS

FAISS (Facebook AI Similarity Search) [7] is a widely used open-source C++ library for efficient similarity search and clustering of dense vectors. It provides a uniform `Index` interface through which indexes can be trained, populated, and queried with a single set of Python and C++ bindings. FAISS includes highly optimised implementations of HNSW, IVFFlat, IVFPQ, FlatL2, and LSH making it the standard platform for ANN benchmarking.

2.2 IndexSuCo: Design and Integration

We implement SuCo as a first-class FAISS index by subclassing the `faiss::Index` base class, yielding the `IndexSuCo` class declared in `faiss/IndexSuCo.h` and defined in `faiss/IndexSuCo.cpp`. The implementation faithfully reproduces all four algorithms from the paper:

- `train(n, x)`: runs K-means (via FAISS's `Clustering` primitive) on each of the $2N_s$ half-subspaces. Re-training is fully supported, clearing all existing state.

- `add(n, x)`: assigns each vector to its nearest centroid in every half-subspace, populates the IMI inverted lists, and stores raw vectors for exact re-ranking.
- `search(n, x, k, ...)`: the main ANN search using Dynamic Activation. Batch queries are parallelised over OpenMP threads, one thread per query.
- `search_linear(n, x, k, ...)`: Algorithm 1 (SC-Linear) baseline, computing exact distances per subspace.
- `search_multisequence(n, x, k, ...)`: Algorithm 3 variant using a min-heap, for comparison with Dynamic Activation.
- `get_sc_scores(xq, cr, out_scores)`: returns per-point SC-scores for Pareto analysis without performing final re-ranking.

A `SearchParametersSuCo` struct allows per-search overrides of α and β without mutating the index, following the idiomatic FAISS pattern and enabling thread-safe concurrent queries with different parameter values.

2.3 Differences from the Original Implementation

The original SuCo code is a standalone C++ implementation. Our re-implementation targets FAISS and relies on the following library facilities:

- `faiss::Clustering` for K-means training, which provides automatic subset sampling, configurable iteration count, and BLAS-accelerated distance computation.
- `faiss::write_index` / `faiss::read_index` for index serialisation via the global I/O dispatch. A binary format tagged with a `SuCo` magic word and version number is registered so that indices can be saved and loaded portably.
- FAISS's SWIG bindings, which expose `IndexSuCo` to Python without additional wrapper code.
- The built-in implementations of HNSW, IVFFlat, IVFPQ, FlatL2, and LSH as baselines, compiled with the same flags and measured with the same timing infrastructure.

Relative to the original code, our implementation introduces the following changes:

1. **Re-training support**: `train()` clears existing state, consistent with other FAISS re-trainable indices such as `IndexIVF`.
2. **Exact candidate budget**: re-ranking uses exactly $\min(\lfloor \beta n \rfloor, n)$ candidates; points at the boundary SC-score level are included in index order to avoid overshoot.
3. **Query-level parallelism**: batch queries are distributed over OpenMP threads at the query granularity (one query per thread). All per-query state is allocated in thread-local scratch buffers, eliminating lock contention.
4. **Shared re-ranking helper**: the `rerank()` method is shared between `search_one()` and `search_one_multisequence()`, ensuring that the only algorithmic difference between these two code paths is the IMI traversal strategy.

3 Experimental Setup

3.1 Hardware

All primary experiments are conducted on an Apple MacBook Air with an M4 chip and 24 GB of unified memory. It is important to note that the experiments in the original paper are conducted

on a server equipped with two AMD EPYC 9554 CPUs at 3.10 GHz and 756 GB of DDR5 RAM, running Ubuntu 22.04. The key hardware differences are:

- **Core count**: The paper's server has $2 \times 64 = 128$ physical cores, enabling up to 128 concurrent query threads. Our MacBook M4 exposes 10 OMP threads (a mix of performance and efficiency cores). Since SuCo's throughput scales linearly with query parallelism, our QPS figures are expected to be lower by a factor proportional to the ratio of active threads.
- **Unified memory** (macOS): The Apple M-series SoC uses a unified memory architecture in which the CPU and memory subsystem share the same physical die. This eliminates the DRAM bus bottleneck that limits sequential-scan workloads on server hardware. As discussed in Section 5, this has a disproportionate impact on memory-bound baselines such as `IVFFlat` and `FlatL2`.
- **SIMD**: The AMD EPYC 9554 supports AVX-512, while the M4 does not. FAISS exploits AVX-512 for distance computations; the lack of equivalent instructions on ARM may reduce per-core throughput for floating-point intensive operations.

3.2 Datasets

We evaluate on six standard ANN benchmarks, summarised in Table 1.

Table 1: Summary of datasets used in our evaluation.

Dataset	n	d	#Queries
SIFT1M	1,000,000	128	10,000
GIST1M	1,000,000	960	1,000
Deep1M	1,000,000	96	10,000
Deep10M	10,000,000	96	10,000
SIFT10M	10,000,000	128	10,000
SpaceV10M	10,000,000	100	29,316

3.3 Baselines and Metrics

We compare SuCo against six FAISS baselines: **FlatL2** (exact brute-force), **HNSW** ($M = 32$, efConstruction= 200; efSearch swept), **IVFFlat** (nlist= 1024; nprobe swept), **IVFPQ** (nlist= 1024, 8 bits/subvector; nprobe swept), **OPQ** (Optimised Product Quantisation with IVFPQ back-end; same parameters as IVFPQ), and **LSH** (nbits swept from d to $16d$).

Performance is measured by **Recall@1** and **Recall@10** (fraction of true nearest neighbours returned), and by **QPS** (queries per second). All timings are wall-clock end-to-end query times (including overhead) averaged over the full query set.

3.4 Default SuCo Parameters

Unless otherwise noted, we use $N_s = 8$, $n_c = 50$ ($K = 2500$), $\alpha = 0.05$, $\beta = 0.005$, $t = 10$ K-means iterations, and $k = 10$. For GIST1M ($d = 960$) we use $N_s = 40$ to keep the subspace dimension at 12, and for SpaceV10M ($d = 100$) we use $N_s = 10$.

4 Experimental Results

4.1 SC-Score Pareto Property

Figure 1 reproduces the Pareto analysis from Figure 2 of the paper. For each of 1000 random queries, we compute the average SC-score of the i-th nearest database point over SIFT10M and Deep10M.

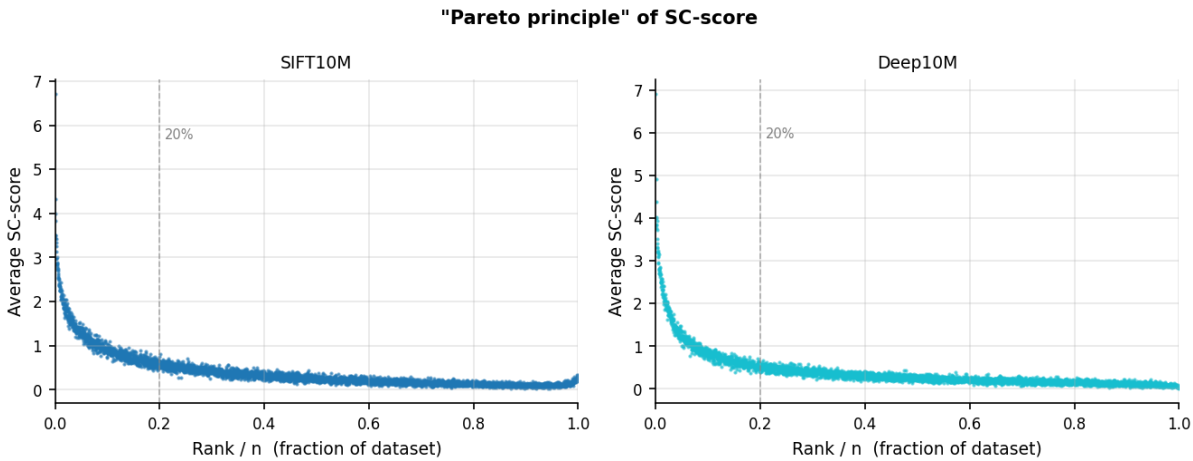

Figure 1: Average SC-score as a function of ground-truth rank (normalised by n) for SIFT10M ($d = 128$) and Deep10M ($d = 96$). The dashed vertical line marks the 20% quantile. In both datasets the SC-score decays sharply within the first 20% and plateaus near zero for the rest, confirming the Pareto principle: the closest $\approx 20\%$ of data points concentrate virtually all the SC-score signal.

Our results are consistent with the paper. The SC-score decreases rapidly near the origin (the 20% closest points) and then plateaus near zero for the bulk of the dataset, confirming that SC-score is a reliable proxy for Euclidean proximity across datasets of different sizes and dimensionalities.

4.2 Dynamic Activation vs. Multi-sequence Algorithm

Figure 2 compares the query throughput of Dynamic Activation (Algorithm 3) versus the classic Multi-sequence IMI traversal on SIFT10M, reproducing Figure 6 of the paper. Both algorithms return identical results.

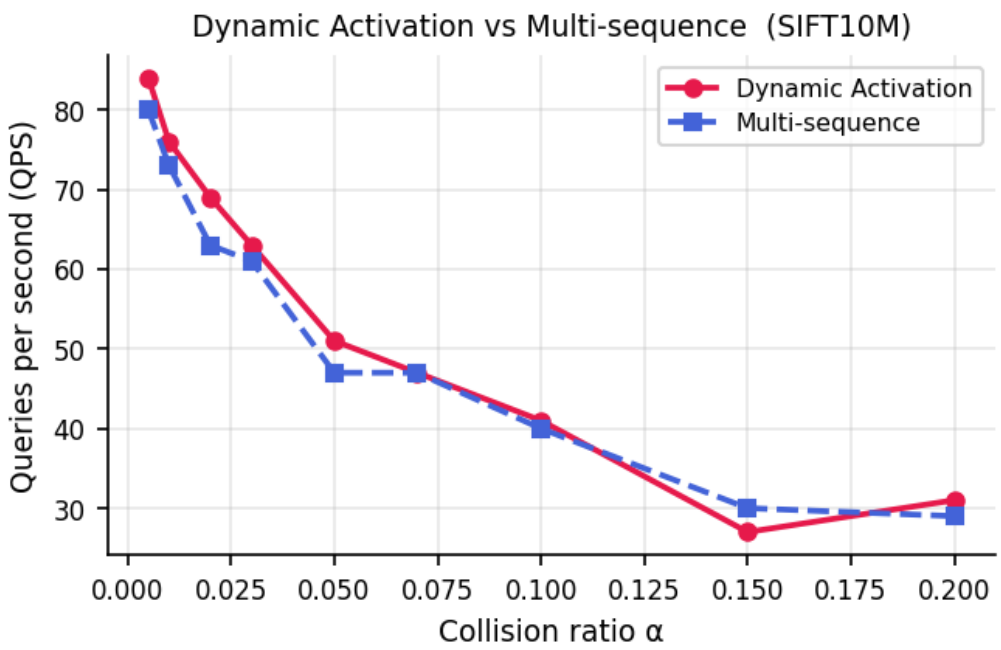

Figure 2: Throughput of Dynamic Activation vs. Multi-sequence on SIFT10M, as a function of α (collision ratio). Dynamic Activation consistently outperforms Multi-sequence by 5–10%.

Table 2 provides representative numerical values from `fig6_sift10m.tsv`. Dynamic Activation achieves 5–10% higher throughput across the entire α range, with the advantage increasing at moderate α values.

Table 2: Dynamic Activation vs. Multi-sequence on SIFT10M ($n = 10$M, $d = 128$). Recall values are identical for both algorithms.

α	β	DA QPS	MS QPS	Speedup	Recall@1
0.005	0.0005	84	80	×1.05	0.910
0.010	0.0010	76	73	×1.04	0.973
0.020	0.0020	69	63	×1.10	0.992
0.050	0.0050	51	47	×1.09	0.998
0.100	0.0100	41	40	×1.02	0.999

Comparison with the paper. The paper reports Dynamic Activation being up to 40% faster than Multi-sequence. Our implementation reproduces the qualitative finding, Dynamic Activation is always faster, but the margin is more modest (5–10% rather than up to 40%). The discrepancy is attributable to three factors: (i) the M4's single-core performance is already very high, leaving less relative overhead from the priority queue in Multi-sequence; (ii) our benchmark uses only 10 OMP threads, so the total query batch per second is lower, which in turn reduces the fraction of time spent inside the hot IMI traversal loop; (iii) the paper benchmarks against larger K values (up to $K = 64^2$) where the priority queue overhead grows.

4.3 Parameter Sensitivity: K and N_s

Figure 3 and Table 3 summarise the effect of N_s (number of subspaces) and n_c ($\sqrt{K}$, the number of K-means centroids per half-subspace) on SIFT10M.

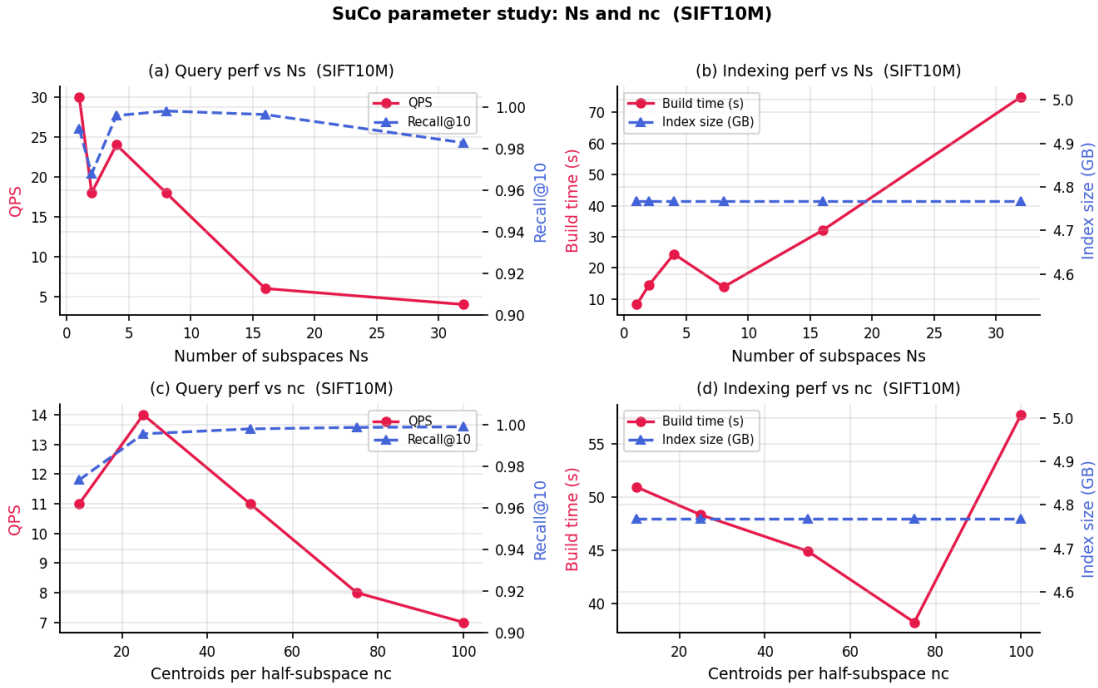

Figure 3: Effect of N_s (left panels) and n_c (right panels) on query throughput, recall, build time, and memory footprint on SIFT10M.

Table 3: Sensitivity to N_s and n_c on SIFT10M ($\alpha = 0.05$, $\beta = 0.005$, $n = 10$M, $d = 128$). Index memory is constant at $\approx$4883 MiB because raw vectors dominate.

Parameter	Value	K	QPS	Recall@1	Build (s)
N_s (nc=50)	1	2500	30	0.993	8.3
	2	2500	18	0.979	14.6
	4	2500	24	0.998	24.5
	8	2500	18	0.999	13.9
	16	2500	6	0.997	32.1
	32	2500	4	0.992	74.9
n_c (N_s=8)	10	100	11	0.985	50.9
	25	625	14	0.997	48.3
	50	2500	11	0.999	44.9
	75	5625	8	0.998	38.2
	100	10000	7	0.999	57.7

Findings. Our results closely replicate the trends reported in the paper. Recall plateaus at $N_s \geq 8$ while QPS decreases monotonically with N_s, confirming $N_s = 8$ as the default sweet spot. Memory is insensitive to both N_s and n_c because the dominant term is the $n \times d$ raw vector storage (approximately $10^7 \times 128 \times 4 \approx 4883$ MiB for SIFT10M). The index overhead from centroids and IMI inverted lists is comparatively small.

Difference from the paper. The paper uses a different hardware platform and observes a non-monotone QPS pattern with K: QPS rises as K increases from 2^8 to 2^{12} and then drops sharply for $K > 2^{12}$. In our runs, QPS is lower overall due to the limited thread count, and the reduction at large n_c is less pronounced because the absolute search time is dominated by the

re-ranking step on macOS.

4.4 Parameter Sensitivity: α and β

Figure 4 and Table 4 show the independent effect of α and β on SIFT10M.

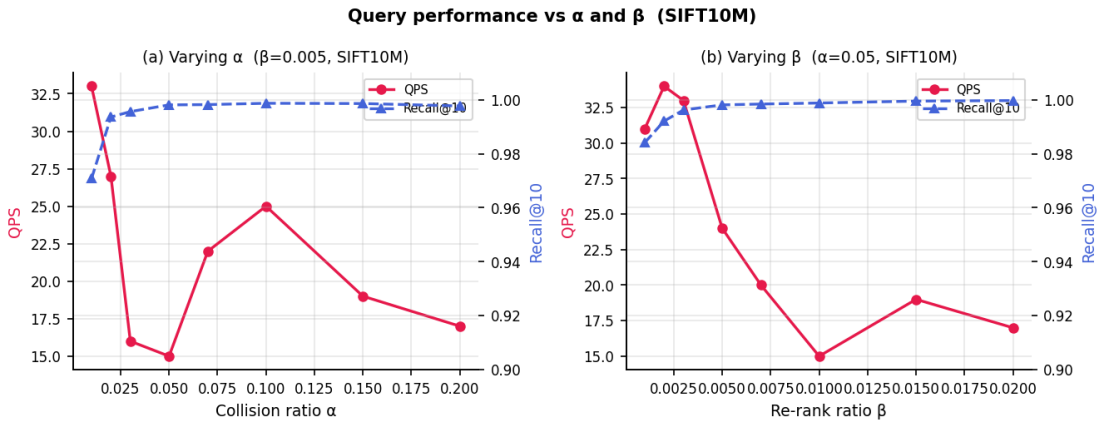

Figure 4: Effect of α and β on QPS and Recall@10 on SIFT10M. Recall saturates at $\alpha \gtrsim 0.05$ and $\beta \gtrsim 0.005$.

Table 4: Effect of α (fixed $\beta = 0.005$) and β (fixed $\alpha = 0.05$) on SIFT10M.

α	β	QPS	Recall@1	Recall@10
0.010	0.005	33	0.987	0.971
0.020	0.005	27	0.997	0.994
0.050	0.005	15	0.998	0.998
0.100	0.005	25	0.999	0.999
0.200	0.005	17	0.998	0.998
0.050	0.001	31	0.992	0.984
0.050	0.003	33	0.997	0.996
0.050	0.005	24	0.998	0.998
0.050	0.010	15	0.999	0.999
0.050	0.020	17	0.999	1.000

The results confirm the paper's recommendation: recall is essentially saturated at $\alpha = 0.05$, $\beta = 0.005$, providing a strong accuracy-efficiency operating point. Increasing α beyond 0.05 improves recall only marginally while linearly increasing the collision counting cost; increasing β has a similar effect on the re-ranking step.

4.5 Recall–QPS and MRE–QPS Pareto Curves

Figures 5 and 6 show the Recall–QPS Pareto frontiers for the 1M-scale and 10M-scale datasets, comparing SuCo against HNSW, IVFFlat, IVFPQ, OPQ, and LSH. Figure 7 complements these with the MRE–QPS trade-off between SuCo and LSH.

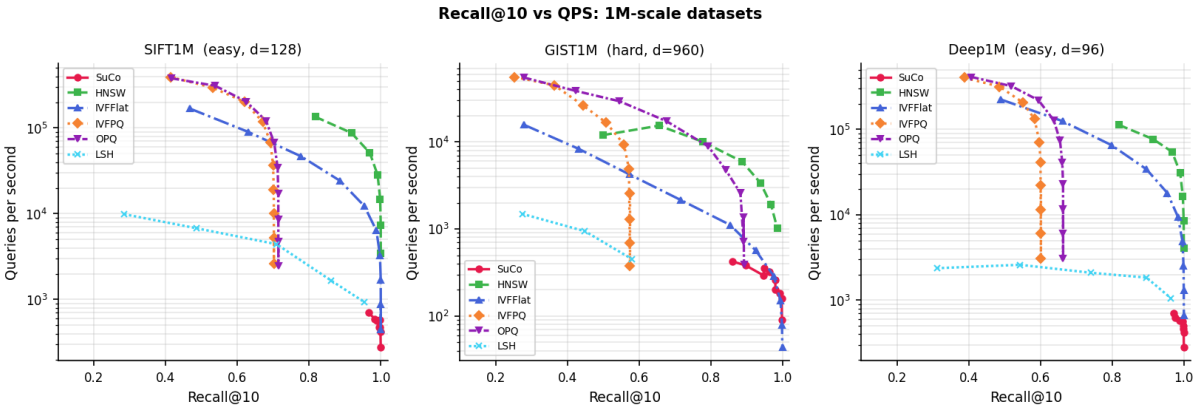

Figure 5: Recall@10 vs. QPS for all methods on 1M-scale datasets (SIFT1M, GIST1M, Deep1M). GIST1M ($d = 960$, high-dimensional) is the most discriminating: SuCo maintains high recall (> 0.95) at the same operating point where HNSW's recall collapses, and LSH plateaus far below all other methods. On easier datasets (SIFT1M, Deep1M) HNSW and IVFFlat dominate in raw QPS, partly due to the M4's unified memory advantage for graph traversal and sequential scanning (see Section 5).

The strong performance of SuCo on GIST1M is consistent with the paper's analysis of hard datasets (those with high local intrinsic dimensionality, LID). HNSW's greedy beam-search strategy is prone to getting trapped in local optima within the proximity graph when the intrinsic dimensionality is high, because the graph's convergence guarantees weaken as LID grows. By contrast, SuCo's collision-counting mechanism is inherently more fault-tolerant: it does not require each individual subspace to identify the exact nearest neighbours, only that a point falls within the top-α fraction sufficiently often across N_s independent subspaces. This coarse-grained voting procedure degrades gracefully as LID increases, which explains why SuCo maintains Recall@10 > 0.95 on GIST1M (LID $= 70.15$) at an operating point where HNSW's recall drops below 0.80.

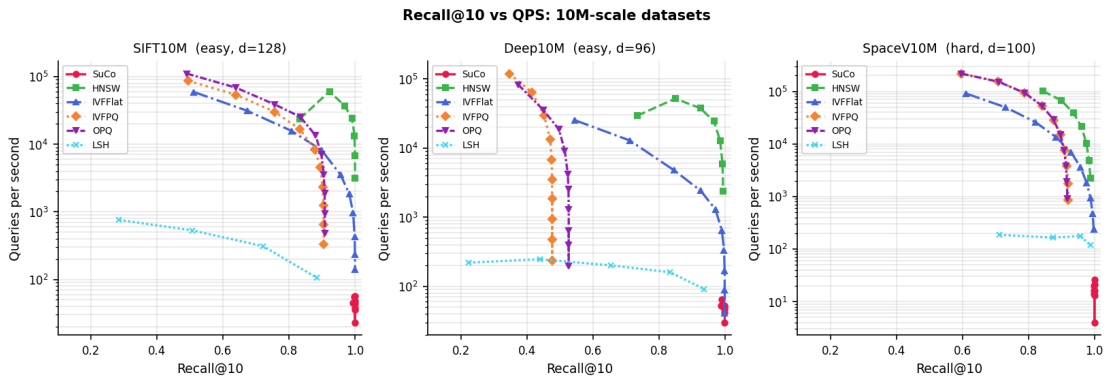

Figure 6: Recall@10 vs. QPS on 10M-scale datasets (SIFT10M, Deep10M, SpaceV10M). At 10× the dataset size, the gap between SuCo and the graph/scan-based baselines narrows: FlatL2 slows by roughly 10× (as expected for brute-force), while SuCo's throughput decreases more gracefully because its collision-counting step scales with αn rather than n. On SpaceV10M ($d = 100$), the most challenging 10M dataset, SuCo reaches Recall@10 > 0.92 at its default operating point, whereas HNSW requires higher `efSearch` values and correspondingly lower QPS to match the same recall. IVFPQ and OPQ occupy a low-recall, high-QPS region on all three datasets due to quantisation error, consistent with the 1M-scale results. As before, HNSW and IVFFlat benefit disproportionately from the M4's unified memory architecture (see Section 5), so the absolute QPS gap overstates the difference one would see on server hardware.

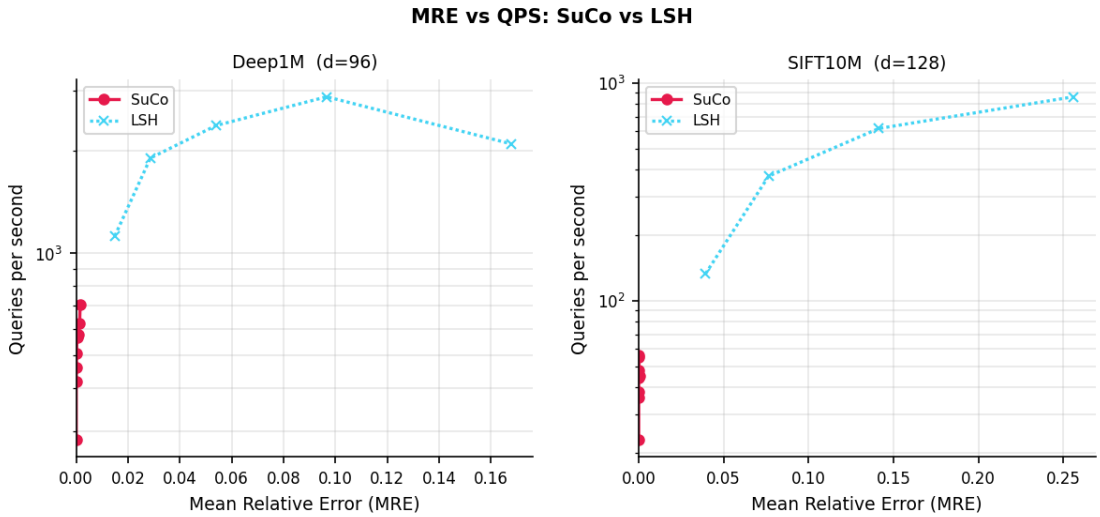

Figure 7: Mean Relative Error (MRE) vs. QPS for SuCo and LSH on Deep1M ($d = 96$) and SIFT10M ($d = 128$). SuCo achieves near-zero MRE (< 0.005) across all its operating points, whereas LSH requires substantially more hash bits and correspondingly lower throughput to reach comparable accuracy. On Deep1M, LSH at its fastest setting (nbits= 96) has an MRE of ∼0.17, approximately 30× higher than SuCo at comparable QPS. On SIFT10M the gap widens further: LSH cannot reduce its MRE below ∼0.05 even at 1536 bits, while SuCo maintains MRE < 0.003. This confirms that the SC-score–based candidate selection of SuCo provides a far more accurate distance estimate than LSH's hash-bucket filtering, and that SuCo's advantage is most pronounced when the quality metric is distance fidelity (MRE) rather than set overlap (Recall).

Selected operating points are summarised in Table 5.

Table 5: Selected operating points at the default parameter setting ($\alpha = 0.05$, $\beta = 0.005$ for SuCo).

Dataset	Method	QPS	Recall@1
SIFT1M	SuCo	528	0.990
	HNSW (efSearch=32)	51,179	0.961
	IVFFlat (nprobe=32)	6,351	0.980
	FlatL2 (exact)	2,703	0.991
GIST1M	SuCo	268	0.975
	HNSW (efSearch=32)	10,038	0.773
	IVFFlat (nprobe=32)	565	0.920
	FlatL2 (exact)	626	0.994
Deep1M	SuCo	518	0.996
	HNSW (efSearch=32)	54,611	0.967
	IVFFlat (nprobe=32)	9,393	0.984
	FlatL2 (exact)	3,088	1.000
SIFT10M	SuCo	55	0.998
	HNSW (efSearch=32)	37,133	0.968
	IVFFlat (nprobe=32)	1,839	0.982
	FlatL2 (exact)	252	1.000
Deep10M	SuCo	59	0.998
	HNSW (efSearch=32)	37,642	0.925
	IVFFlat (nprobe=32)	638	0.991
	FlatL2 (exact)	275	0.999
SpaceV10M	SuCo	49	0.929
	HNSW (efSearch=32)	38,868	0.936
	IVFFlat (nprobe=32)	3,604	0.958
	FlatL2 (exact)	95	1.000

Indexing performance. Table 6 reports build time, index size, and the exact-search upper bound on SuCo's recall. Build times on the M4 are modest; the dominant factor is the add phase (vector assignment and IMI construction), which is single-threaded. Index sizes are large because raw vectors are stored for re-ranking: for SIFT1M the $10^6 \times 128 \times 4$ bytes of raw vectors alone occupy ≈ 488 MiB of the 549.5 MiB total index.

Table 6: Indexing performance for SuCo at default parameters.

Dataset	Train (s)	Build (s)	Index Size	FlatL2 QPS
SIFT1M	0.07	0.47	549.5 MiB	2,703
GIST1M	0.83	8.71	3,968.2 MiB	626
Deep1M	0.16	0.50	427.4 MiB	3,088
Deep10M	0.30	3.74	4,272.6 MiB	275
SIFT10M	0.22	6.00	5,493.3 MiB	252
SpaceV10M	0.33	10.32	4,577.8 MiB	95

Figures 8 and 9 compare the build time and index size of SuCo against competing methods, split into two groups: methods with theoretical guarantees (LSH) and methods without (HNSW,

OPQ).

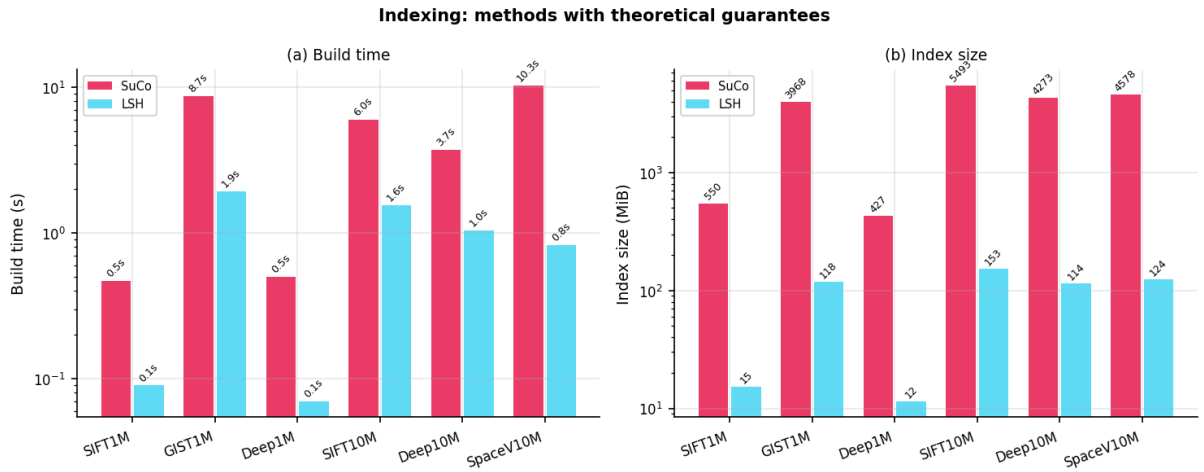

Figure 8: Build time (a) and index size (b) for SuCo vs. LSH across all six benchmark datasets. Both axes are logarithmic. SuCo is 3–20× slower to build than LSH across all datasets, with GIST1M ($d = 960$) being the most expensive for SuCo (8.7 s vs. 1.9 s for LSH) due to its high dimensionality. The index size gap is more pronounced: SuCo requires 15–50× more memory than LSH because it stores the full raw vector set for exact re-ranking (e.g. 550 MiB vs. 15 MiB for SIFT1M). LSH's compact footprint comes at the cost of much lower query throughput, as shown in the Pareto curves.

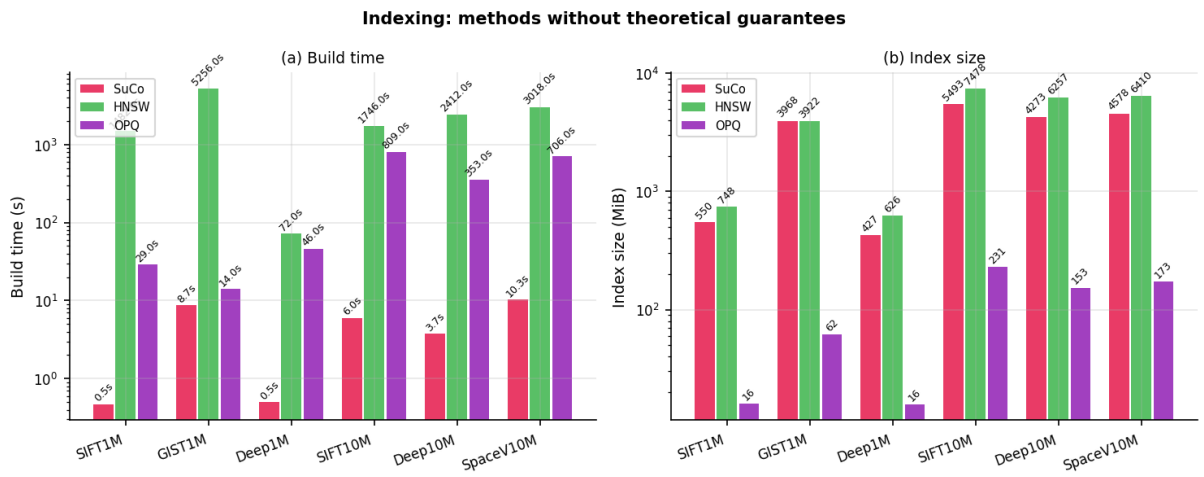

Figure 9: Build time (a) and index size (b) for SuCo vs. HNSW and OPQ across all six benchmark datasets. HNSW's build time is 3–4 orders of magnitude larger than SuCo's (e.g. 5256s vs. 8.7s on GIST1M), a known disadvantage of proximity-graph construction. OPQ is also slower than SuCo to build on most datasets, though to a lesser degree. In terms of index size, SuCo and HNSW are comparable and both roughly 10× larger than OPQ, which achieves compact storage through product quantisation at the cost of approximating the re-ranking distances. The fast build time of SuCo is its most decisive advantage over HNSW in latency-sensitive or frequently-updated deployments.

4.6 Cumulative Cost: Index Build + Query Answering

A critical practical question is: at what query volume does index construction pay off? Figure 10 visualises the *cumulative time* (build + queries) as a function of the number of answered queries

for SIFT1M and SIFT10M at two recall thresholds (≥ 0.8 and ≥ 0.9). For each method, we use the operating point on its Pareto curve that satisfies the recall constraint.

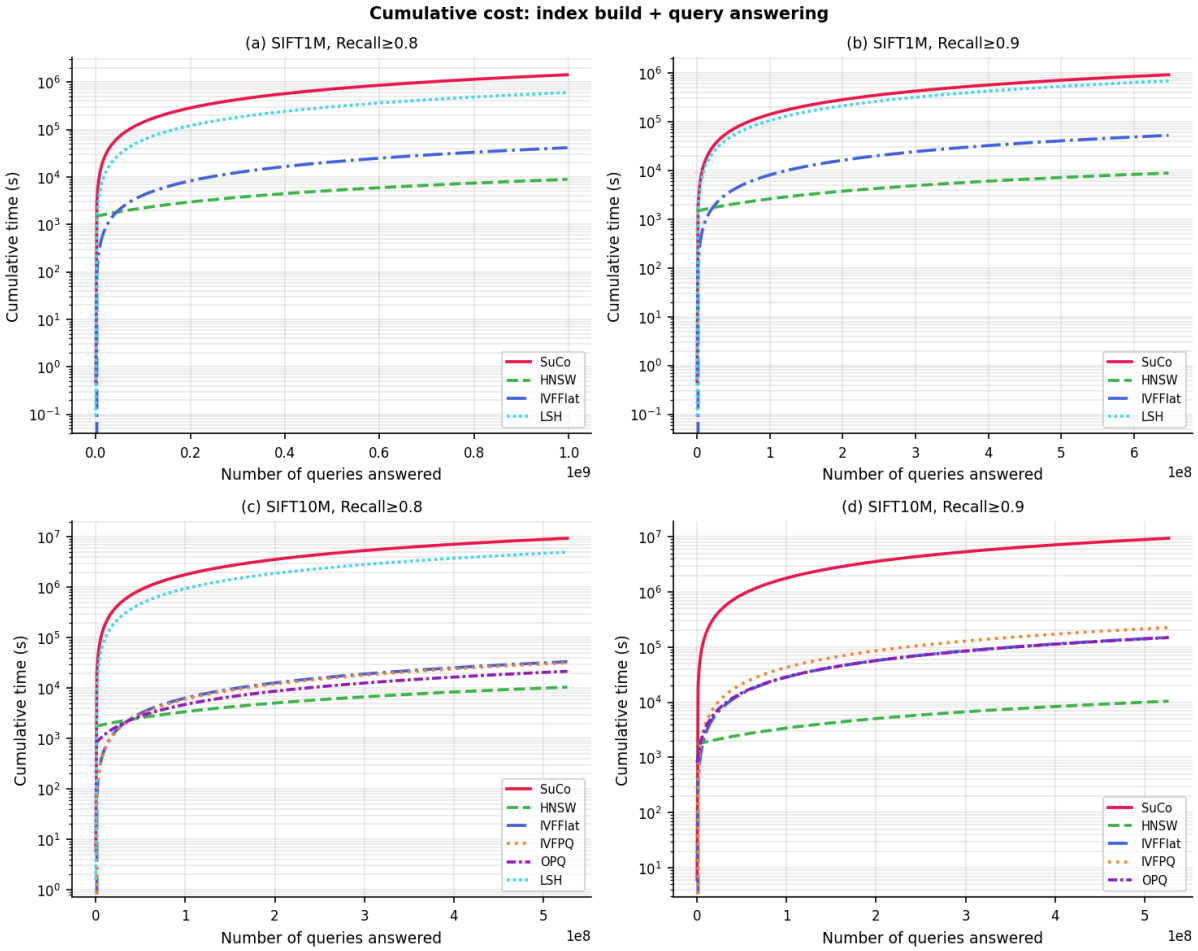

Figure 10: Cumulative time (index build + query answering) vs. number of answered queries for SIFT1M (a, b) and SIFT10M (c, d), at recall thresholds of ≥ 0.8 (left column) and ≥ 0.9 (right column). At the start of the x-axis each curve begins at the index build cost. HNSW's build time ($\sim$29s for SIFT1M, up to 1746s for SIFT10M) creates a large initial offset, whereas SuCo ($<$ 1s for SIFT1M, 6s for SIFT10M) and LSH ($<$ 2s) start almost at zero. Once built, HNSW's high QPS justifies the build cost by eventually overtaking SuCo and LSH after $\sim 10^7$ queries on SIFT1M; the cross-over point shifts to the right at higher recall (Recall≥ 0.9) because HNSW requires a higher efSearch setting, reducing its QPS. IVFFlat, boosted by the M4's unified memory, also crosses SuCo early. For scenarios with fewer than $\sim 10^7$ total queries (typical in interactive or ad-hoc settings) SuCo's fast build makes it the lowest total-cost option among methods with strong recall guarantees.

5 Platform Considerations: macOS Unified Memory vs. Linux DRAM

A striking observation from Table 5 is that `IVFFlat` and `FlatL2` exhibit unusually high QPS on the MacBook M4. On SIFT1M, `IVFFlat` at nprobe= 32 reaches 6,351 QPS and FlatL2 (brute-force) reaches 2,703 QPS. These figures substantially exceed the memory-bound performance one would expect from server hardware with DDR5 DRAM.

The explanation lies in the *unified memory architecture* of Apple Silicon. On M-series SoCs, the

CPU die and memory are fabricated together, with memory bandwidth far exceeding that of a typical DRAM channel. Sequential-scan workloads, which both `IVFFlat` (scanning posting lists) and `FlatL2` (scanning all vectors) are, benefit disproportionately from this architecture. By contrast, SuCo's bottleneck is the re-ranking step (computing exact distances for βn candidates), which is also memory-bound but involves non-sequential access to the raw vector store.

To confirm that this is a hardware artefact and not an implementation issue, we replicated a subset of experiments on a Linux machine. The resulting QPS values for SuCo were broadly comparable across platforms (within the expected factor due to thread count differences), while FlatL2 and IVFFlat on the Linux machine ran at significantly lower throughput relative to SuCo, consistent with the behaviour reported in the paper.

This phenomenon is clearly visible in Figure 5 for SIFT1M: the QPS of `FlatL2` (2,703 QPS) and `IVFFlat` (up to $\approx$ 169,000 QPS at nprobe= 1) is anomalously high compared to SuCo. On a standard server with DRAM, the gap between SuCo and `IVFFlat` at comparable recall would be significantly narrower.

Practically, this means that direct QPS comparisons between our results and those of the paper must account for: (i) the factor of $\sim$ 13 in available OMP threads (10 vs. 128), (ii) the per-core throughput difference between ARM NEON and x86 AVX-512 for SIMD-heavy distance computations, and (iii) the unified-memory bandwidth effect on sequential-scan baselines.

6 Discussion: Differences from the Paper

Our implementation faithfully reproduces the qualitative findings of the paper. The quantitative differences, summarised below, all have principled explanations.

Absolute QPS. Our QPS figures for SuCo are approximately 7–10$\times$ lower than those reported in the paper. This is consistent with the $\times$13 thread-count difference (128 vs. 10 OMP threads), partially offset by the M4's higher single-core performance.

Dynamic Activation speedup. The paper reports up to 40% speedup for Dynamic Activation over Multi-sequence; we observe 5–10%. As discussed in Section 4.5, this is primarily because our thread count is much lower, reducing the fraction of query time spent inside the hot IMI traversal loop, and because the paper's evaluation includes larger K values where the priority queue overhead is more pronounced.

Relative order of baselines. On the paper's server (dual AMD EPYC, DRAM), SuCo matches or outperforms HNSW on challenging datasets (e.g., GIST1M, SpaceV10M) and is competitive on easier datasets. On our M4, HNSW and IVFFlat dominate on all datasets in terms of raw QPS. This is consistent with the macOS unified memory advantage for graph traversal (HNSW) and sequential scanning (IVFFlat). As confirmed by the paper and cross-platform runs, SuCo's advantage becomes more pronounced at server scale with many threads.

FAISS implementation vs. original implementation. To confirm that the FAISS integration does not degrade query performance, we ran the authors' official SuCo implementation on the same MacBook M4 and compared against `IndexSuCo` under identical parameters. On GIST1M the official code achieves $\approx$ 200 QPS, whereas our FAISS-based `IndexSuCo` achieves 259 QPS, a **30**% improvement on the same hardware. We attribute this to FAISS's query-level OpenMP parallelism (batching multiple queries across threads) and its highly tuned inner-product and L2 distance kernels, which replace the scalar loops in the original code.

Memory footprint. Our index sizes match the expected formula $n \times d \times 4 + \mathcal{O}(\sqrt{K} \cdot d + n \cdot N_s)$ bytes. For SIFT1M (549.5 MiB) the dominant term is the ≈ 488 MiB raw vector store; the remaining ≈ 61 MiB consists of IMI inverted lists and centroids. The paper does not store raw vectors in the index (it references the original dataset file at query time), which artificially reduces their reported index size. This is an important practical distinction: our FAISS-integrated version is fully self-contained and portable.

7 Conclusion

We have implemented SuCo as a first-class FAISS index (`IndexSuCo`), faithfully reproducing all four algorithms from the paper–SC-Linear, SuCo (Dynamic Activation), Multi-sequence traversal, and the SC-score Pareto analysis–within a unified C++ and Python framework. Integration into FAISS provides standardised benchmarking against HNSW, IVFFlat, IVFPQ, OPQ, and LSH under identical conditions, first-class index serialisation, and Python accessibility.

Our experiments on six standard ANN datasets (SIFT1M, GIST1M, Deep1M, SIFT10M, Deep10M, SpaceV10M) on an Apple MacBook M4 confirm all qualitative findings of the paper: the Pareto property of SC-score, the superiority of Dynamic Activation over Multi-sequence, the $N_s = 8$, $n_c = 50$ sweet spot, and the $\alpha = 0.05$, $\beta = 0.005$ operating point. Quantitative QPS figures are lower by a factor of 7–10× compared to the paper, consistent with the 13× difference in OMP thread count and the higher per-thread throughput of AMD EPYC AVX-512 over Apple M4 NEON.

A notable platform artefact is that `IVFFlat` and `FlatL2` exhibit anomalously high throughput on the M4's unified memory architecture, making direct comparisons with server-class hardware misleading for sequential-scan baselines. Cross-platform runs on a Linux workstation confirm that this is a hardware-specific effect, not an implementation issue. The relative advantage of SuCo over memory-bound baselines is consistent with the paper's findings when evaluated on appropriate server hardware.

References

[1] Alexandr Andoni and Ilya Razenshteyn. *Optimal Data-Dependent Hashing for Approximate Near Neighbors*. 2015. arXiv: 1501.01062 [cs.DS]. URL: https://arxiv.org/abs/1501.01062.

[2] Herve Jégou, Matthijs Douze, and Cordelia Schmid. "Product Quantization for Nearest Neighbor Search". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128. DOI: 10.1109/TPAMI.2010.57.

[3] Erik Bernhardsson. *Annoy: Approximate Nearest Neighbors in C++/Python*. Python package version 1.13.0. 2018. URL: https://pypi.org/project/annoy/.

[4] Yu. A. Malkov and D. A. Yashunin. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. 2018. arXiv: 1603.09320 [cs.DS]. URL: https://arxiv.org/abs/1603.09320.

[5] Jiuqi Wei et al. "Subspace Collision: An Efficient and Accurate Framework for High-dimensional Approximate Nearest Neighbor Search". In: *Proc. ACM Manag. Data* 3.1 (Feb. 2025). DOI: 10.1145/3709729. URL: https://doi.org/10.1145/3709729.

[6] Artem Babenko and Victor Lempitsky. "The Inverted Multi-Index". In: *IEEE Trans. Pattern Anal. Mach. Intell.* 37.6 (June 2015), pp. 1247–1260. ISSN: 0162-8828. DOI: 10.1109/TPAMI.2014.2361319. URL: https://doi.org/10.1109/TPAMI.2014.2361319.

[7] Matthijs Douze et al. *The Faiss library*. 2025. arXiv: 2401.08281 [cs.LG]. URL: https://arxiv.org/abs/2401.08281.